

Module 13: Operator Overloading

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

A lighter, more mechanical module after two conceptually dense ones - a good place to let the pace breathe a little. The core message is restraint: operator overloading is genuinely useful for types that behave like values (numbers, quantities), and genuinely harmful when applied to types where the operator's meaning doesn't transfer naturally. Make that judgment call, not just the syntax, a first-class part of the lesson.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Overload a binary operator (+) as a class method.
- Overload a comparison operator (==) as a class method.
- Overload << as a friend free function so std::cout can print a custom object directly.
- Explain why << cannot be a regular member method.
- Judge when an operator's built-in meaning genuinely fits a custom type, and when it doesn't.

Prerequisites

- Module 7 (function overloading) and Module 8 (classes) from Part 1 - this module is a direct extension of both ideas.

Materials & Instructor Prep

- 13-operator-overloading/examples/ preloaded - all three are short and worth running in one sitting for a sense of the whole pattern.

Key Vocabulary

Term	Definition
<code>operator+</code> / <code>operator==</code> / <code>operator<<</code>	Special method names the compiler recognizes and translates infix syntax (<code>a + b</code>) into (<code>a.operator+(b)</code>).
<code>const</code> parameter / <code>const</code> method	A promise that a value won't be modified - the standard, expected shape for an overloaded operator's parameter and, usually, the method itself.
friend function	A free (non-member) function granted access to a class's private members - used here because <code><<</code> must be written outside the class it prints.

Lesson Outline & Instructor Talking Points

1. Motivate with readability, not just syntax (10 min)

- Show `a1.add(a2)` vs `a1 + a2` side by side. Ask: 'which one would you rather read in a long calculation with several terms?' Let the class articulate the readability win themselves.
- Preview the domain payoff explicitly for the EE track: 'AC circuit impedance IS complex-number arithmetic - by the end of this module you can write `Z1 + Z2` and mean it literally.'

2. `operator+` (15 min)

- Live-build `Complex::operator+` from `examples/01_operator_plus.cpp`. Pause on ``a.operator+(b)`` as the literal translation of ``a + b`` - write both forms on the board side by side.
- Highlight `const Complex& other` and the trailing `const` on the method. Ask: 'why reference, not by value?' (avoid an unnecessary copy - Module 7's principle, still applying) and 'why `const`?' (we only read `other`, and marking the whole method `const` lets the compiler catch us if we ever accidentally try to modify `a`.)

3. `operator==` (10 min)

- Run `examples/02_operator_equals.cpp`. Keep this brief - it's the same shape as `operator+`, just returning `bool`.

4. `operator<<` - the exception to the pattern (20 min)

- Ask directly: 'why can't we write `operator<<` as a method of `Complex`, the same way we did `operator+`?' Give the class a moment before answering.
- Reveal: `std::cout << c` would need to be `c`'s method called on `cout`, backwards - `<<` must be written from `cout`'s side, and we don't own `std::ostream`. Introduce the friend free-function pattern as the standard fix.
- Frame this explicitly as a template to copy and adapt, not something to derive from first principles every time - the shape (friend, take/return `std::ostream&`) is always the same.

5. Judgment: which operators are worth overloading? (10–15 min)

- Pose a deliberately bad example: overloading + on an Employee class to mean 'merge two unrelated employee records.' Ask why this is a bad idea even though it compiles fine.
- Land the rule: overload an operator only when its built-in meaning genuinely transfers to your type. When in doubt, a clearly named method beats a misleading operator.

Instructor note: *This judgment discussion is worth protecting time for - it's the difference between students who overload operators thoughtfully later and students who overload everything because it's newly available to them.*

Common Student Misconceptions

Misconception	How to Address It
"operator+ should modify one of the two operands directly."	It should return a NEW object; a + b should never change a or b themselves - exactly like built-in arithmetic never changes its operands.
"<< can be overloaded as a normal member method, just like + and ==."	The left-hand side of << is a std::ostream, not your class - it must be a free function (typically a friend) written from the stream's side.
"If C++ lets me overload an operator for any meaning I want, that's fine to do."	Technically true, practically harmful - an operator whose behavior doesn't match its conventional meaning actively misleads readers. Prefer a named method when the fit isn't natural.

Exercises & Module Project

Exercise: 13-operator-overloading/exercises/exercise1.cpp - a Fraction class overloading +, ==, and <<. Project (three tracks): Generic - Vector2D; EE - Complex Impedance (+, -, * for AC circuit analysis); Biomedical - Dosage (+, *, < for safe dose combination/scaling/comparison).

Facilitation guidance: The EE track's Complex class is a genuine, not just illustrative, real-world use case - if you have EE-track students, spend a moment connecting $Z_{\text{total}} = Z_1 + Z_2$ back to actual circuit theory they may already know from a physics or circuits course.

Discussion & Comprehension Check Questions

1. Why must operator<< be written as a free function instead of a member method?

Guidance: The left operand of << is a std::ostream (like std::cout), which your class doesn't own - the function must be written from the stream's side, typically granted friend access to reach your

class's private data.

2. Give an example of an operator overload that would be confusing rather than helpful, and explain why.

Guidance: Any overload whose behavior doesn't match the operator's conventional meaning - e.g. + meaning something other than 'combine' misleads a reader's correct intuition.

Assessment Ideas

- Code review exercise: give students a class with a questionable operator overload (e.g. * meaning something unrelated to multiplication) and have them argue for or against it in writing.

Answer Key Notes

See [13-operator-overloading/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Every program so far forgets everything the moment it closes. Next module gives your programs memory that survives - reading and writing files."

Instructor Tips

- This is a good module to let run slightly under time if earlier modules ran long - the core pattern (operator+, ==, <<) is short and the judgment discussion can be trimmed to a quick poll if needed without losing the module's core value.